DWES-06-ApiFunkosConjunta

Álvaro Ruiz Velázquez // Adrián García-Heras

DESARROLLO WEB EN ENTORNOS SERVIDOR

Índice

1. Introducción 5
1.1. Descripción general del proyecto 5
1.2. Objetivos del proyecto 5
2. Descripción y funcionamiento de la aplicación 6
2.1. Funcionamiento general 6
2.2. Recursos gestionados 6
2.3. Operaciones disponibles 6
2.4. Autenticación y permisos 7
2.5. Imágenes de funkos 7
2.6. Datos iniciales 7
3. Arquitectura del proyecto 7
3.1. Arquitectura general 7
3.2. Responsabilidades de las capas 8
3.3. Interfaces e inyección de dependencias 8
3.4. Estructura de directorios 9
4. Modelos, DTO y mapeo de datos 11
4.1. Modelo Funko 11
4.2. Modelo Categoría 11
4.3. Modelo Proveedor 11
4.4. DTO 12
4.5. Mappers 12
5. Persistencia de datos 12
5.1. PostgreSQL y Entity Framework Core 12
5.2. Configuración de las entidades 13
5.3. MongoDB 13
5.4. Relación entre motores 13
5.5. Inicialización del esquema 13

6. Relaciones y reglas de negocio 14
6.1. Cardinalidad 14
6.2. Creación y actualización de funkos 14
6.3. Eliminación de categorías y proveedores 14
6.4. Coordinación mediante CatalogoService 14
7. API REST y endpoints 15
7.1. Rutas de Funkos 15
7.2. Rutas de Categorías 15
7.3. Rutas de Proveedores 15
7.4. Autenticación 16
7.5. Filtros y paginación 16
7.6. Ejemplo de creación 16
8. Autenticación y autorización 17
8.1. Obtención del token 17
8.2. Contenido y duración del token 17
8.3. Restricción de operaciones 17
9. Sistema de caché Redis 17
9.1. Integración 17
9.2. Datos almacenados 18
9.3. Estrategia de lectura 18
9.4. Escrituras 18
10. Validación y gestión de errores 18
10.1. Validación de entrada 18
10.2. Errores de dominio 18
10.3. Patrón Result 19
10.4. Notificaciones 19
11. Contenerización con Docker 19
11.1. Objetivo 19
11.2. Servicios 19
11.3. Red interna 20
11.4. Volúmenes 20
12. Pruebas automatizadas 20

12.1. Proyecto de pruebas 20
12.2. Herramientas 20
13. Decisiones de diseño 21
13.1. Objetivo de la contenerización 21
13.2. Docker Compose 21
13.3. PostgreSQL y Redis 21
13.4. Volúmenes y Healthchecks 21
14. Pruebas 21
14.1. Proyecto de pruebas 21
14.2. NUnit, Moq y FluentAssertions 22
14.3. Resultados obtenidos 22
15. Decisiones de diseño 22
15.1. Separación de responsabilidades 22
15.2. Servidor coordinador 22
15.3. Persistencia híbrida 22
15.4. Redis 23
15.5. DTO y Mappers 23
15.6. Docker 23
16. Flujos de funcionamiento 23
16.1. Consulta de listado 23
16.2. Consulta por identificador 24
16.3. Creación de Funko 24
16.4. Actualización de Funko 25
16.5. Eliminación de categoría o proveedor 25
16.6. Invalidación de caché 25
17. Funcionalidades adicionales de la API 26
17.1. Documentación interactiva con Swagger/OpenAPI 26
17.2. Política CORS 26
17.3. Consultas GraphQL 26
18. Notificaciones en tiempo real con SignalR 26
18.1. Hub y autenticación 26
18.2. Grupos y eventos 27

19. Servicio de correo electrónico 27
19.1. Componentes 27
19.2. MailDev 27
20. Configuración de entornos y ejecución 27
20.1. Requisitos previos 27
20.2. Development 28
20.3. Production 28
20.4. Separación y persistencia 28
20.5. Variables de entorno y seguridad 28
21. Inicialización, dependencias y pruebas 29
21.1. Seeders y comportamiento por entorno 29
21.2. Inyección de dependencias 29
21.3. Pruebas y ejecución 29
21.4. Comandos de pruebas y cobertura 29
22. Conclusión 30

1. Introducción

1.1. Descripción general del proyecto

ApiFunkosConjunta es una API desarrollada con C# y ASP.NET Core sobre .NET 10 para gestionar un catálogo de figuras Funko y la información relacionada con sus categorías y proveedores. Expone endpoints REST para consultar, crear, actualizar y eliminar registros, y amplía su funcionalidad con Swagger/OpenAPI, consultas GraphQL mediante HotChocolate, notificaciones en tiempo real con SignalR y envío de correos electrónicos mediante MailKit.

Una característica principal del proyecto es el uso combinado de distintos sistemas de persistencia: PostgreSQL almacena funkos y categorías, MongoDB almacena proveedores y Redis se utiliza como caché distribuida para acelerar las consultas.

1.2. Objetivos del proyecto

- Desarrollar una API REST con ASP.NET Core y .NET 10.
- Implementar CRUD independientes para Funkos, categorías y proveedores.
- Persistir Funkos y categorías en PostgreSQL y proveedores en MongoDB.
- Generar identificadores de categoría de 11 caracteres con el generador de formato YouTube.
- Utilizar ObjectId como identificador de los proveedores.
- Representar la relación entre Funkos, categorías y proveedores y validar su integridad desde la capa de aplicación.
- Incorporar filtros por nombre, búsqueda por identificador y paginación.
- Guardar la ruta relativa de la imagen de cada Funko.
- Integrar Redis como caché distribuida e invalidar los datos almacenados cuando se producen escrituras.
- Proteger las operaciones de escritura mediante JWT y el rol Administrador.
- Aplicar separación de responsabilidades, inyección de dependencias y resultados tipados.
- Comprobar el comportamiento mediante pruebas unitarias, Testcontainers y peticiones Bruno.
- Documentar y probar la API con Swagger/OpenAPI.
- Ofrecer consultas GraphQL mediante HotChocolate.
- Enviar notificaciones de Funkos en tiempo real con SignalR.
- Incorporar el envío de correos con MailKit y procesamiento en segundo plano.
- Mantener configuraciones separadas de Development y Production mediante Docker Compose.

2. Descripción y funcionamiento de la aplicación

2.1. Funcionamiento general

La aplicación recibe peticiones HTTP a través de sus controladores. Estos delegan las operaciones en CatalogoService, que actúa como coordinador de las operaciones del catálogo. El servicio consulta primero la caché cuando se trata de lecturas; si no encuentra el dato, delega en el servicio y repositorio correspondientes para acceder al motor de persistencia.

En las operaciones de escritura, la lógica comprueba los datos y las relaciones necesarias, realiza la operación sobre el repositorio y actualiza o invalida las entradas de caché afectadas.

2.2. Recursos gestionados

Recurso	Identificador	Persistencia	Responsabilidad
Funko	int autoincremental	PostgreSQL	Figura del catálogo, stock, precio, categoría, proveedor e imagen
Categoría	string (11 chars)	PostgreSQL	Clasificación de los Funkos
Proveedor	MongoDB ObjectID	MongoDB	Datos del proveedor: nombre, contacto y país

2.3. Operaciones disponibles

- Consulta de listados con filtro opcional por nombre y paginación.
- Consulta individual por identificador.
- Creación de registros con comprobación de datos y relaciones.
- Actualización de registros existentes.
- Eliminación, restringida cuando existen Funkos asociados a la categoría o proveedor.

2.4. Autenticación y permisos

Las consultas GET son públicas. Las operaciones POST, PUT y DELETE requieren un token JWT válido cuyo rol sea Administrador. El token se obtiene mediante el endpoint de inicio de sesión. La API valida el token con el middleware de autenticación Bearer configurado en la aplicación.

2.5. Imágenes de funkos

La propiedad Imagen contiene una ruta relativa, por ejemplo imagenes/funkos/spider-man.png. La base de datos no almacena el contenido binario de la imagen ni una URL absoluta; conserva únicamente la referencia relativa.

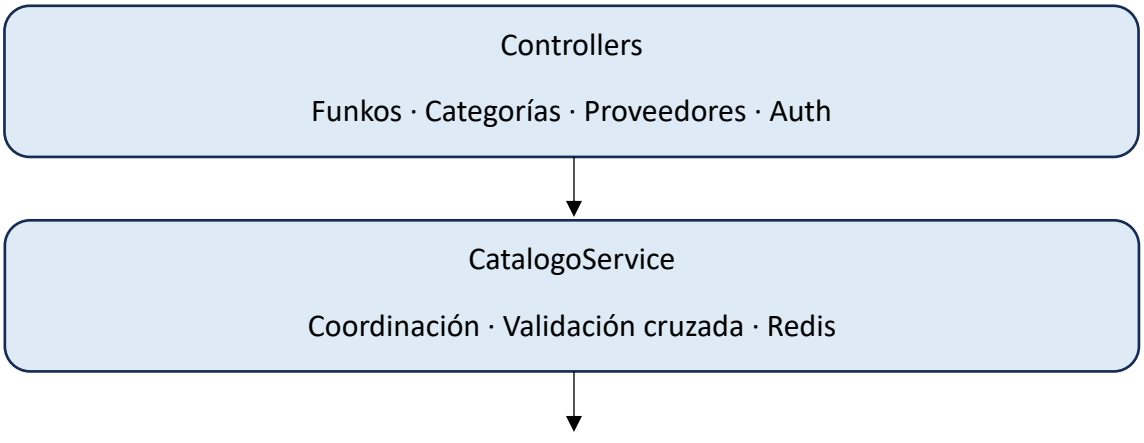
2.6. Datos iniciales

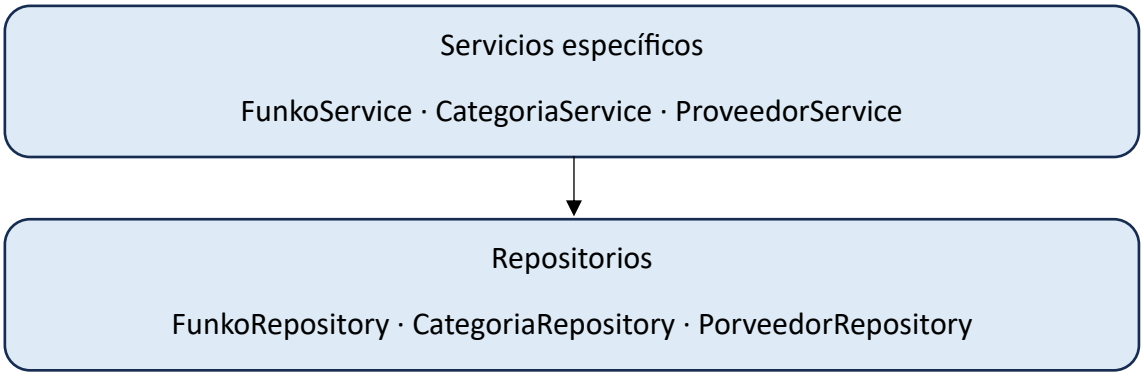
Durante el arranque se ejecutan seeders de categorías, usuarios, proveedores y Funkos. En Development se reinicializan las bases de datos y Redis antes de sembrar los datos para disponer de un estado de pruebas conocido; en Production se conservan los datos existentes y los seeders se ejecutan sin borrar previamente la información.

3. Arquitectura del proyecto

3.1. Arquitectura general

El proyecto utiliza una arquitectura por capas con responsabilidades diferenciadas. Los controladores reciben y responden a HTTP; el servicio coordinador aplica las reglas transversales; los servicios específicos contienen la lógica de cada recurso; y los repositorios encapsulan el acceso a cada sistema de persistencia.





3.2. Responsabilidades de las capas

- Controllers: definen las rutas HTTP, reciben DTO y convierten los resultados en códigos de estado y respuestas.
- CatalogoService: coordina las operaciones del catálogo, valida relaciones entre entidades y gestiona la caché Redis.
- Services: contienen la lógica específica de cada entidad, autenticación y correo electrónico; también incluyen el procesamiento de envíos en segundo plano.
- Repositories: encapsulan el acceso a PostgreSQL o MongoDB.
- Models: representan las entidades persistidas.
- Dto: define los contratos de entrada, salida y los objetos usados para caché.
- Mappers: convierten entre modelos y DTO.
- Infrastructure: centraliza el registro de dependencias y la configuración de persistencia, CORS, Swagger y correo electrónico.
- Errors: contiene los errores de dominio, servicio, repositorio, autenticación y API.
- GraphQL: contiene las consultas y los tipos de respuesta de GraphQL.
- Hubs: contiene el hub de SignalR para notificaciones de Funkos.

3.3. Interfaces e inyección de dependencias

Los repositorios y servicios específicos exponen interfaces para desacoplar sus consumidores de las implementaciones concretas. La configuración se centraliza en Infrastructure/DependencyInjection.cs y Infrastructure/Persistence/PersistenceExtensions.cs. Los controladores de catálogo dependen de ICatalogoService, mientras que el controlador de autenticación utiliza IAuthService.

3.4. Estructura de directorios

```
ApiFunkosConjunta/
├─── .run/
├─── ApiFunkosConjunta/
│    ├─── Common/
│    │    ├─── Configuration/
│    │    ├─── Examples/
│    │    ├─── Http/
│    │    ├─── Json/
│    │    └─── Pagination/
│    ├─── Controllers/
│    ├─── Data/
│    │    ├─── Configurations/
│    │    └─── Seed/
│    ├─── Dto/
│    │    ├─── Auth/
│    │    ├─── Categorias/
│    │    ├─── Funkos/
│    │    └─── Proveedores/
│    ├─── Errors/
│    ├─── GraphQL/
│    │    ├─── Queries/
│    │    └─── Types/
│    ├─── Hubs/
│    ├─── IdGenerators/
│    ├─── Infrastructure/
│    │    ├─── Filters/
│    │    └─── Persistence/
│    ├─── Mappers/
│    ├─── Models/
│    ├─── Repositories/
│    │    ├─── Implementations/
│    │    └─── Interfaces/
│    ├─── Services/
```

```
│   │   ├── Email/
│   │   ├── Implementations/
│   │   └── Interfaces/
│   └── Properties/
├── ApiFunkosConjunta.Tests/
│   ├── Common/
│   │   ├── Configuration/
│   │   ├── Examples/
│   │   ├── Http/
│   │   ├── Json/
│   │   └── Pagination/
│   ├── Controllers/
│   │   ├── Auth/
│   │   ├── Categorias/
│   │   ├── Funkos/
│   │   └── Proveedores/
│   ├── Data/
│   │   └── Seed/
│   ├── Errors/
│   ├── GraphQL/
│   │   └── Queries/
│   ├── Hubs/
│   ├── Infrastructure/
│   │   └── Filters/

│   ├── Mappers/
│   │   ├── Categorias/
│   │   └── Proveedores/
│   ├── Repositories/
│   │   ├── Categorias/
│   │   ├── Funkos/
│   │   └── Proveedores/
│   ├── Services/
│   │   ├── Auth/
│   │   ├── Email/
```

```
│   │   ├── Funkos/
│   │   ├── Categorias/
└── └── └── Proveedores/
```

4. Modelos, DTO y mapeo de datos

4.1. Modelo Funko

Propiedad	Tipo	Descripción
Id	int	Clave primaria autoincremental
Nombre	string	Nombre de la figura
CategoriaId	string	Identificador de la categoría relacionada
ProveedorId	string	Identificador del proveedor relacionado
Stock	int	Unidades disponibles
Precio	decimal	Precio con precisión decimal
Imagen	string	Ruta relativa de la imagen

4.2. Modelo Categoría

Propiedad	Tipo	Descripción
Id	string	Clave primaria generada con YouTubeIdValueGenerator
Nombre	string	Nombre de la categoría

4.3. Modelo Proveedor

Propiedad	Tipo	Descripción
Id	ObjectId	Clave primaria nativa de MongoDB

Nombre	string	Nombre del proveedor
Contacto	string	Email de contacto
País	string	País de origen del proveedor

4.4. DTO

Los DTO separan los datos que recibe o devuelve la API de las entidades internas. Existen DTO de creación y actualización, DTO de respuesta y DTO específicos para almacenar los datos serializados en Redis. Las carpetas se organizan por recurso dentro de Dto.

- Funkos: CrearFunkoDto, ActualizarFunkoDto, FunkoDto y CacheFunkoDto.
- Categorías: CrearCategoriaDto, ActualizarCategoriaDto, CategoriaDto y CacheCategoriaDto.
- Proveedores: CrearProveedorDto, ActualizarProveedorDto, ProveedorDto y CacheProveedorDto.
- Autenticación: LoginDto y LoginResponseDto.

4.5. Mappers

Las clases FunkoMapper, CategoriaMapper y ProveedorMapper concentran las conversiones entre entidades y DTO. De esta forma, los controladores y servicios no necesitan repetir manualmente la asignación de propiedades.

5. Persistencia de datos

5.1. PostgreSQL y Entity Framework Core

PostgreSQL almacena las tablas funkos y categorias. El acceso se realiza mediante Entity Framework Core y el proveedor Npgsql. AppDbContext representa el contexto relacional, mientras que las configuraciones de entidad se encuentran en Data/Configurations. Los repositorios encapsulan las operaciones de persistencia y se consumen mediante sus interfaces.

5.2. Configuración de las entidades

- La tabla de categorías utiliza una clave primaria de tipo cadena generada por el generador personalizado.
- La tabla de Funkos utiliza un identificador entero autoincremental.
- El nombre de Funko es obligatorio y tiene una longitud máxima de 100 caracteres.
- La relación Funko-Categoria se implementa mediante una clave externa.
- El borrado de una categoría relacionada se restringe para evitar dejar Funkos sin categoría.
- El precio de Funko utiliza precisión decimal (10,2).
- La imagen se guarda como ruta relativa con longitud máxima configurada.

5.3. MongoDB

MongoDB almacena los documentos de proveedores en la colección proveedores. El driver oficial MongoDB.Driver proporciona el cliente, la base de datos y la colección inyectados en el repositorio.

5.4. Relación entre motores

La referencia a proveedor no puede implementarse como una clave externa relacional porque el proveedor se almacena en MongoDB. Por ello, Funko.ProveedorId conserva el identificador como texto y el servicio coordinador comprueba su existencia consultando el servicio de proveedores.

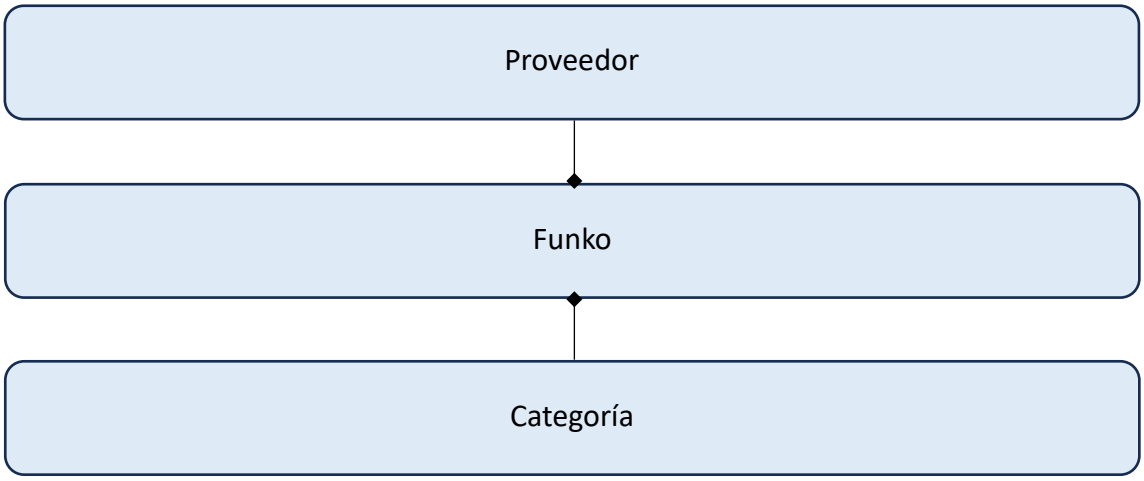
La integridad entre PostgreSQL y MongoDB se aplica desde la lógica de negocio; no existe una transacción distribuida ni una clave externa SQL entre ambos motores.

5.5. Inicialización del esquema

En el entorno Development, el inicializador de categorías utiliza EnsureCreatedAsync() para crear el esquema cuando corresponde. Esta versión del proyecto no utiliza migraciones de Entity Framework Core.

6. Relaciones y reglas de negocio

6.1. Cardinalidad



- Un Funko pertenece a un proveedor y a una categoría.
- Un proveedor puede suministrar varios Funkos.
- Una categoría puede clasificar varios Funkos.

6.2. Creación y actualización de funkos

Antes de guardar un Funko, CatalogoService comprueba que la categoría indicada exista en PostgreSQL y que el proveedor indicado exista en MongoDB. Si alguna referencia no es válida, la operación devuelve un error de dominio y no se realiza la escritura.

6.3. Eliminación de categorías y proveedores

El servicio coordinador comprueba si hay Funkos que referencian la categoría o el proveedor. Si existen dependencias, la eliminación se rechaza para mantener la integridad lógica del catálogo.

6.4. Coordinación mediante CatalogoService

CatalogoService es el punto de entrada de las operaciones de catálogo desde los controladores. Delega el CRUD específico en CategoriaService, FunkoService y ProveedorService, y concentra las reglas que afectan a más de una entidad, además de la estrategia de caché.

7. API REST y endpoints

7.1. Rutas de Funkos

Método	Ruta	Acceso
GET	/api/funkos	Público
GET	/api/funkos/{id}	Público
POST	/api/funkos	Administrador
PUT	/api/funkos/{id}	Administrador
DELETE	/api/funkos/{id}	Administrador

7.2. Rutas de Categorías

Método	Ruta	Acceso
GET	/api/categorias	Público
GET	/api/categorias /{id}	Público
POST	/api/categorías	Administrador
PUT	/api/categorias/{id}	Administrador
DELETE	/api/categorias/{id}	Administrador

7.3. Rutas de Proveedores

Método	Ruta	Acceso
GET	/api/proveedores	Público
GET	/api/proveedores/{id}	Público
POST	/api/proveedores	Administrador

PUT	/api/proveedores/{id}	Administrador
DELETE	/api/proveedores/{id}	Administrador

7.4. Autenticación

Método	Ruta	Acceso
POST	/api/auth/login	Público

7.5. Filtros y paginación

Los endpoints de listado aceptan los parámetros nombre, página y tamanoPagina. La página por defecto es 1, el tamaño por defecto es 10 y el máximo permitido es 100. La respuesta utiliza PagedResult<T> e incluye los datos, el total de registros, la página y el tamaño de página.

GET /api/funkos?nombre=batman&pagina=1&tamanoPagina=10

7.6. Ejemplo de creación

POST /api/funkos
Authorization: Bearer JWT_GENERADO
Content-Type: application/json

```
{
  "nombre": "Iron Man",
  "categoriaId": "ID_CATEGORIA_EXISTENTE",
  "proveedorId": "64b000000000000000000001",
  "stock": 15,
  "precio": 29.99,
  "imagen": "imagenes/funkos/iron-man.png"
}
```

El identificador de categoría y el ObjectId del proveedor deben corresponder a registros existentes. Los nombres exactos de las propiedades de entrada se definen en los DTO de creación.

8. Autenticación y autorización

8.1. Obtención del token

El endpoint POST /api/auth/login recibe un LoginDto. AuthService comprueba el usuario y la contraseña configurados bajo la sección Jwt. Si las credenciales son correctas, genera un JWT firmado mediante HMAC-SHA256.

POST /api/auth/login
Content-Type: application/json

```
{
  "username": "admin",
  "password": "Admin123!"
}
```

8.2. Contenido y duración del token

- Identidad del usuario (claim Name).
- Rol Administrador.
- Emisor y audiencia configurados.
- Expiración de dos horas.

8.3. Restricción de operaciones

Los métodos GET no requieren autenticación. Los métodos POST, PUT y DELETE incorporan [Authorize(Roles = "Administrador")]. Para las operaciones protegidas, el cliente debe enviar el token en la cabecera Authorization.

Authorization: Bearer JWT_GENERADO

9. Sistema de caché Redis

9.1. Integración

Redis se registra como caché distribuida mediante AddStackExchangeRedisCache. CatalogoService recibe IDistributedCache por inyección de dependencias, por lo que la lógica de aplicación trabaja con la abstracción de .NET.

9.2. Datos almacenados

- Listados paginados de Funkos, categorías y proveedores.
- Consultas individuales de Funkos, categorías y proveedores.

9.3. Estrategia de lectura

Se aplica una estrategia Cache-Aside: primero se consulta Redis. Si existe una entrada válida, se deserializa y devuelve. Si no existe, se consulta el servicio o repositorio correspondiente, se serializa el resultado y se almacena en caché. Las entradas tienen una expiración absoluta de cinco minutos.

9.4. Escrituras

Tras crear, actualizar o eliminar un registro, se invalida la caché de la entidad correspondiente. Los listados utilizan claves versionadas por entidad; al cambiar los datos se actualiza la versión y las siguientes consultas generan claves nuevas. También se elimina la entrada individual del registro modificado o eliminado. Esta estrategia evita recorrer y borrar una a una todas las combinaciones de filtros y paginación.

10. Validación y gestión de errores

10.1. Validación de entrada

Los DTO aplican atributos de validación de .NET, como Required, StringLength, Range y RegularExpression cuando corresponde. Las reglas comprueban campos obligatorios, longitudes, valores numéricos e identificadores.

10.2. Errores de dominio

La carpeta Errors agrupa tipos de error específicos para la API, los servicios, los repositorios, la autenticación y la coordinación del catálogo. Esta organización permite describir los fallos esperados sin depender de excepciones como mecanismo ordinario de control del flujo.

10.3. Patrón Result

Los servicios utilizan CSharpFunctionalExtensions y devuelven resultados tipados, habitualmente Result<T, DomainErrors>. El resultado permite distinguir entre éxito y fallo, y transportar el error que explica la operación no completada.

10.4. Notificaciones

Las notificaciones de las operaciones sobre Funkos se envían en tiempo real mediante SignalR. El hub permite dirigir los eventos a los administradores y al usuario que realiza la operación; los errores de envío se registran sin impedir que finalice la persistencia.

11. Contenerización con Docker

11.1. Objetivo

Docker Compose permite levantar la API y los servicios de infraestructura con una configuración común, evitando tener que instalar PostgreSQL, MongoDB y Redis directamente en el equipo.

11.2. Servicios

Servicio	Imagen	Puerto
apifunkosconjunta	Construida desde Dockerfile	5000
postgresql	postgres:18	5432
mongodb	mongo:8	27017
redis	redis:8	6379
maildev-web	MailDev (interfaz web)	1080 Development / 1081 Production
maildev-smtp	MailDev (SMTP)	1025 Development

11.3. Red interna

Dentro de la red creada por Compose, la API se conecta a los servicios mediante sus nombres: postgres, mongodb y redis. El puerto interno de la API es 8080 y se publica en el puerto 5000 del host.

11.4. Volúmenes

Se definen volúmenes persistentes para PostgreSQL, MongoDB y Redis. Estos conservan los datos al detener y volver a iniciar los contenedores. Los entornos Development y Production utilizan proyectos Compose, redes y volúmenes independientes.

12. Pruebas automatizadas

12.1. Proyecto de pruebas

El proyecto independiente ApiFunkosConjunta.Tests contiene pruebas de controladores, servicios, repositorios, mappers y errores.

12.2. Herramientas

Herramienta	Uso
NUnit	Framework de ejecución y organización de pruebas
Moq	Creación de dobles de prueba para aislar dependencias
FluentAssertions	Aserciones expresivas y legibles
TestContainers.PostgreSql	Contenedor PostgreSQL para pruebas de repositorios
TestContainers.MongoDb	Contenedor MongoDB para pruebas de repositorios
coverlet.collector	Recogida de información de cobertura

13. Decisiones de diseño

13.1. Objetivo de la contenerización

Aislar la aplicación y sus dependencias en contenedores ligeros y portables para garantizar ejecuciones idénticas en cualquier entorno (desarrollo, testing, producción).

13.2. Docker Compose

Los archivos compose.yaml, compose.local.yaml y compose.prod.yaml definen los servicios necesarios para la API y su infraestructura. compose.local.yaml permite ejecutar la infraestructura localmente y compose.prod.yaml levanta la API contenerizada junto con PostgreSQL, MongoDB, Redis y MailDev.

13.3. PostgreSQL y Redis

- PostgreSQL: Servicio configurado sobre la imagen oficial postgres:alpine, especificando credenciales, nombre de base de datos y puerto 5432.
- Redis: Servicio sobre redis:alpine exponiendo el puerto 6379.

13.4. Volúmenes y Healthchecks

- Volúmenes: Montados para persistir los datos de PostgreSQL y Redis entre reinicios de contenedores.
- Healthchecks: Verificaciones de estado en PostgreSQL y Redis para asegurar que la aplicación .NET espere a que la infraestructura esté totalmente operativa antes de iniciar.

14. Pruebas

14.1. Proyecto de pruebas

Se dispone del proyecto ApiFunkosConjunta.Tests, configurado como un proyecto de pruebas unitarias y de integración independiente dentro de la misma solución.

14.2. NUnit, Moq y FluentAssertions

- NUnit: Framework ejecutor de las pruebas unitarias ([TestFixture], [Test]).
- Moq: Herramienta para crear objetos simulados de las interfaces (Mock<IUserRepository>, Mock<IUserCache>, Mock<IJsonPlaceholderApi>).
- FluentAssertions: Biblioteca que proporciona una sintaxis expresiva y legible para las aserciones (ej: result.IsSuccess.Should().BeTrue()).

14.3. Resultados obtenidos

Todas las pruebas unitarias automatizadas se ejecutan con éxito, garantizando una alta cobertura del código (code coverage) y previniendo regresiones durante futuras refactorizaciones.

15. Decisiones de diseño

15.1. Separación de responsabilidades

La separación entre controladores, servicios y repositorios evita que la capa HTTP dependa directamente de la persistencia. Cada componente mantiene una responsabilidad delimitada.

15.2. Servidor coordinador

CatalogoService centraliza las reglas que afectan a varias entidades y la caché, sin asumir el acceso directo a DbContext ni a los repositorios. Delega el trabajo específico en los servicios correspondientes.

15.3. Persistencia híbrida

PostgreSQL ofrece almacenamiento relacional para Funkos y categorías. MongoDB almacena los proveedores con ObjectId. Esta elección responde al requisito de utilizar distintos modelos de persistencia en el mismo proyecto.

15.4. Redis

IDistributedCache permite trabajar con una abstracción común para la caché. Las claves versionadas simplifican la invalidación de los listados paginados y filtrados.

15.5. DTO y Mappers

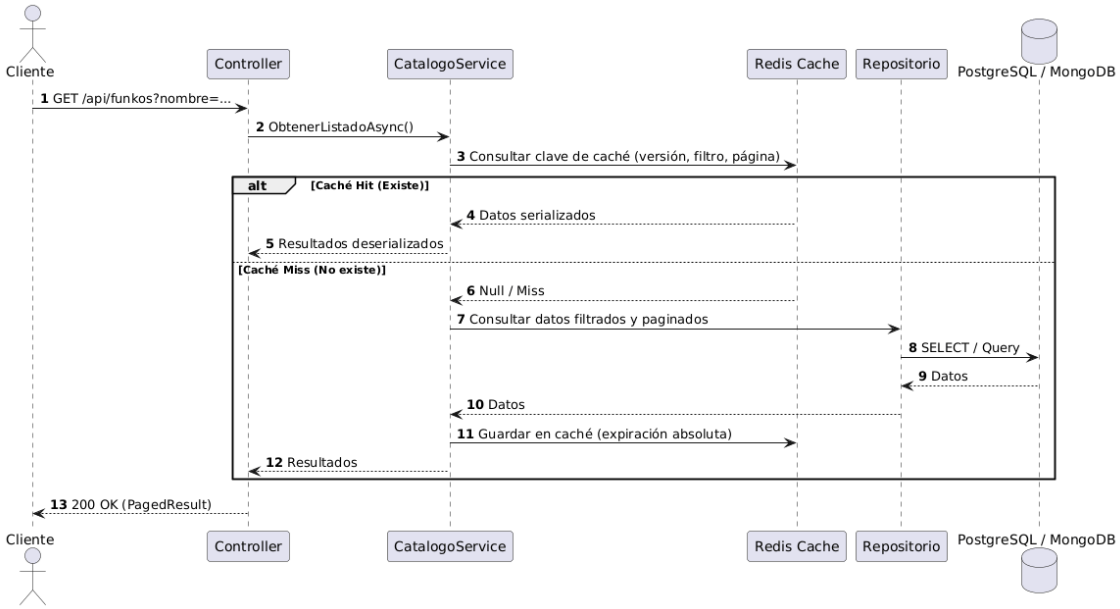
Los DTO definen contratos explícitos de entrada y salida y los mappers concentran la conversión entre las representaciones HTTP y las entidades.

15.6. Docker

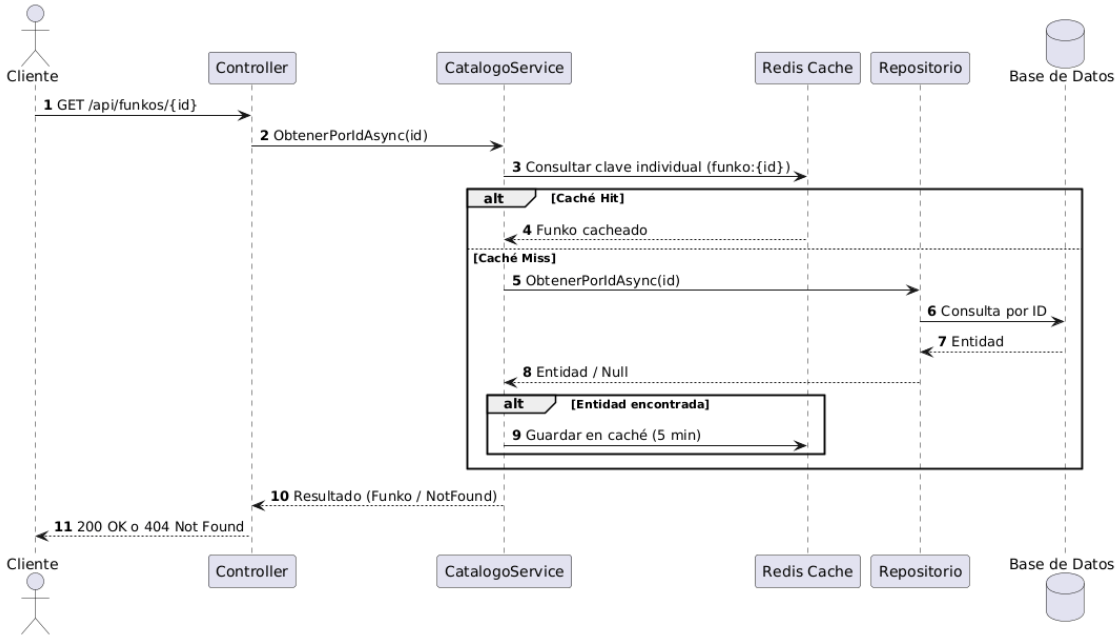
Docker Compose facilita levantar los servicios externos de la solución con versiones e identificadores de servicio definidos en un único fichero.

16. Flujos de funcionamiento

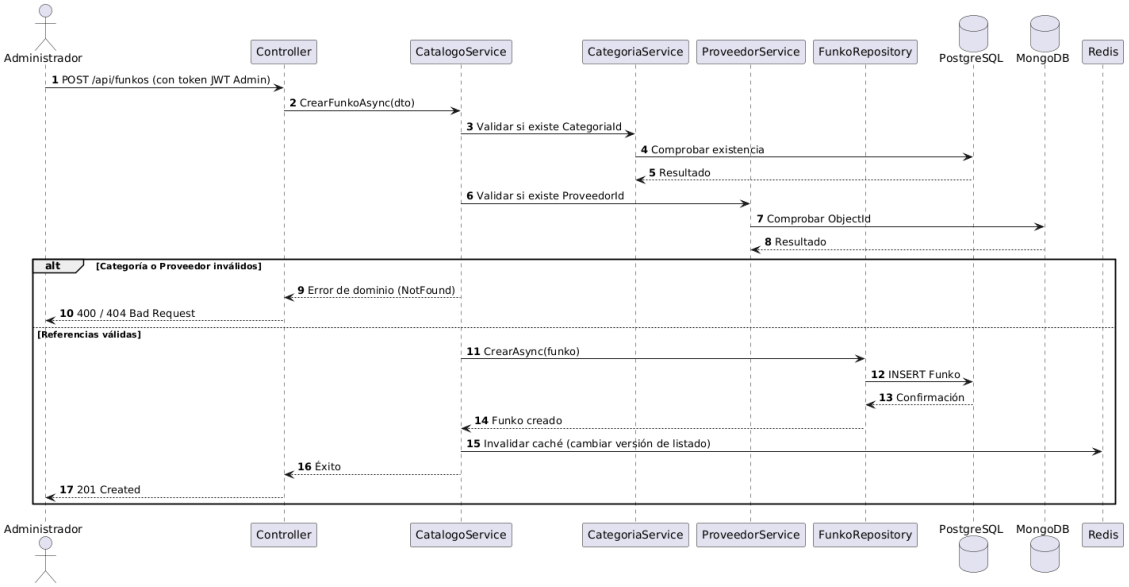
16.1. Consulta de listado



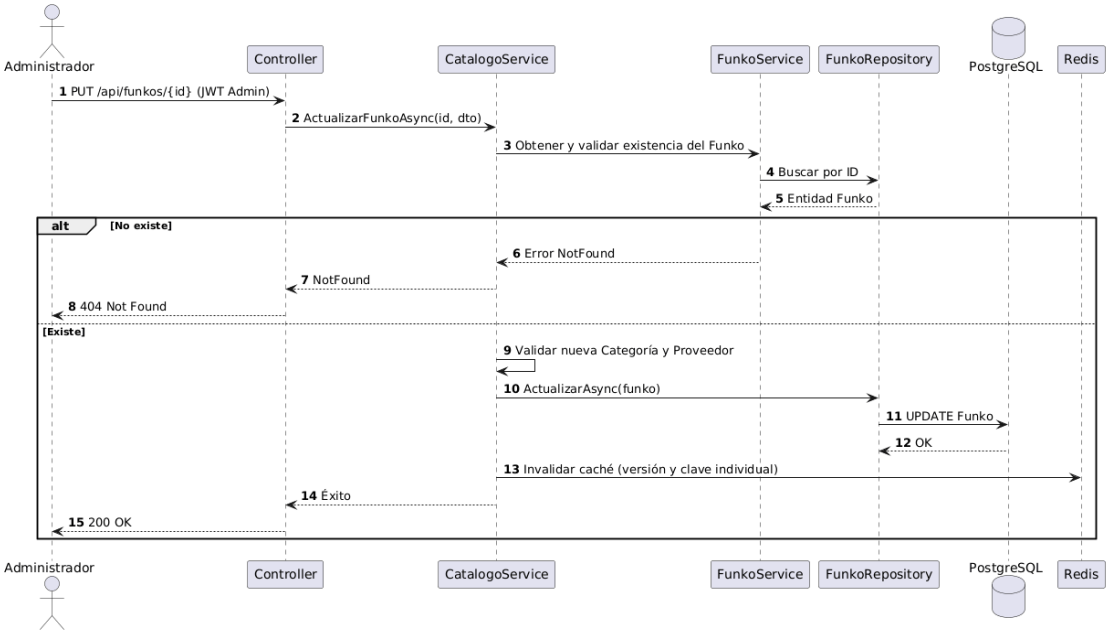
16.2. Consulta por identificador



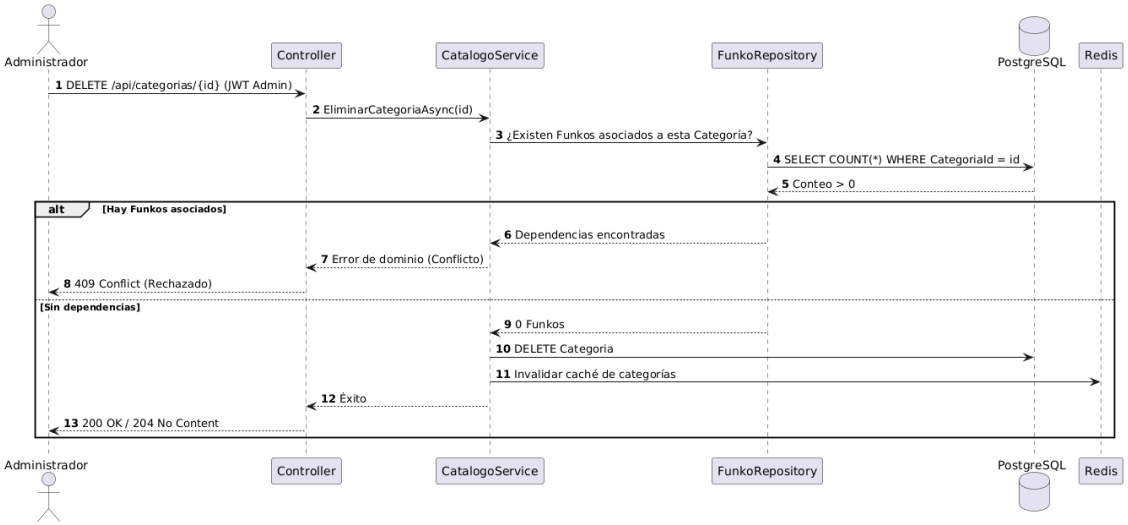
16.3. Creación de Funko



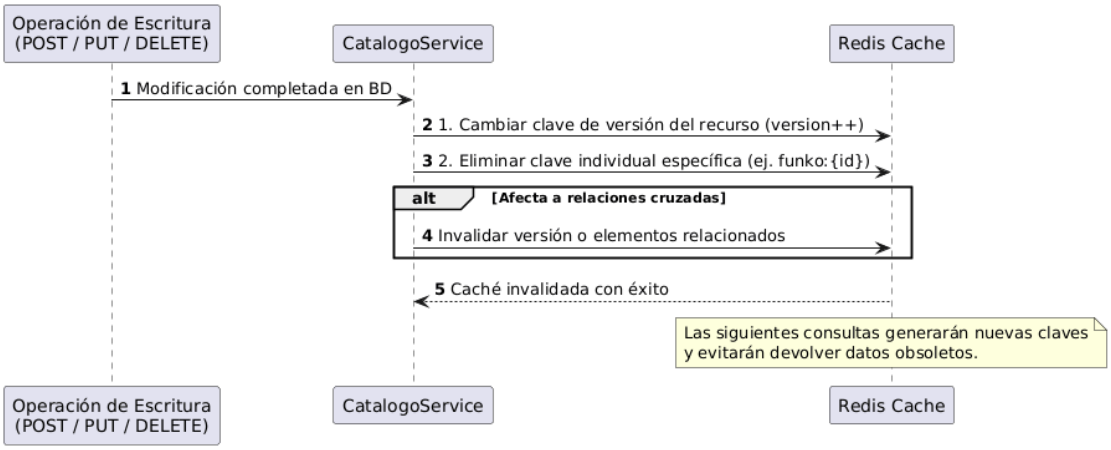
16.4. Actualización de Funko



16.5. Eliminación de categoría o proveedor



16.6. Invalidación de caché



17. Funcionalidades adicionales de la API

17.1. Documentación interactiva con Swagger/OpenAPI

La API utiliza Swashbuckle para publicar la documentación OpenAPI y la interfaz Swagger UI. En Development, Swagger UI está disponible en http://localhost:5087/swagger y el documento JSON en http://localhost:5087/swagger/v1/swagger.json. La configuración se encuentra en Infrastructure/SwaggerConfig.cs e incorpora el esquema Bearer para JWT, comentarios XML, anotaciones, ejemplos de petición y ejemplos de ProblemDetails. Desde Swagger UI se puede ejecutar el login, copiar el token y utilizar Authorize para probar los endpoints protegidos.

17.2. Política CORS

La política CORS se configura en Infrastructure/CorsConfig.cs. En Development se utiliza AllowAll, que permite cualquier origen, método y cabecera. En Production se aplica ProductionPolicy y solo se permiten los orígenes indicados en Cors:AllowedOrigins, configurables mediante appsettings.Production.json o variables de entorno como Cors__AllowedOrigins__0 y Cors__AllowedOrigins__1.

17.3. Consultas GraphQL

La API incorpora GraphQL mediante HotChocolate 16. El endpoint /graphql está disponible en Development y Production. Nitro, la interfaz gráfica de exploración, se habilita únicamente en Development. La implementación ofrece consultas (Queries), sin Mutations ni Subscriptions: estado para comprobar el servicio, funkos para obtener listados filtrados y paginados, y funko(id) para consultar un Funko individual. Las consultas reutilizan los servicios existentes y no duplican las reglas de negocio.

18. Notificaciones en tiempo real con SignalR

18.1. Hub y autenticación

SignalR permite notificar en tiempo real las operaciones realizadas sobre Funkos. El hub FunkoHub está disponible en /hubs/funkos y requiere autenticación JWT. La configuración de JwtBearer permite recibir el token mediante el parámetro access_token cuando la conexión se realiza al hub.

18.2. Grupos y eventos

Al establecer la conexión, el hub incorpora al usuario al grupo usuario-{usuarioId}. Si el usuario tiene el rol Administrador, también se incorpora al grupo administradores. Los eventos publicados son FunkoCreado, FunkoActualizado y FunkoEliminado. Las notificaciones se dirigen a los grupos correspondientes según la operación. Los errores de envío se registran y no impiden completar la persistencia del catálogo.

19. Servicio de correo electrónico

19.1. Componentes

El envío de correo se implementa mediante MailKit y mantiene separadas la generación, gestión y entrega de los mensajes. MailKitEmailService realiza el envío SMTP; MemoryEmailService proporciona una implementación en memoria; EmailBackgroundService procesa mensajes en segundo plano; EmailTemplates centraliza las plantillas; EmailConfig y SmtpSettings agrupan la configuración SMTP. Las credenciales se suministran mediante configuración o variables de entorno, no directamente en el código.

19.2. MailDev

Durante el desarrollo se utiliza MailDev como servidor SMTP local y visor de mensajes. La interfaz web está disponible en http://localhost:1080 y el servidor SMTP en localhost:1025. En la configuración de demostración de Production, la interfaz web se publica en http://localhost:1081. Esto permite comprobar los mensajes sin depender de un servidor de correo externo.

20. Configuración de entornos y ejecución

20.1. Requisitos previos

Se necesita .NET 10 SDK, Docker Desktop o Docker Engine con Docker Compose, un entorno de desarrollo compatible con .NET (por ejemplo, Rider o Visual Studio) y Bruno para ejecutar las colecciones de peticiones. No es necesario instalar PostgreSQL, MongoDB ni Redis directamente, ya que Compose proporciona esos servicios.

20.2. Development

El entorno de desarrollo levanta la infraestructura en Docker y permite ejecutar la API directamente desde Rider. Desde la raíz se ejecuta: docker compose -f compose.local.yaml up -d. Se inician PostgreSQL (5432), MongoDB (27017), Redis (6379), MailDev web (1080) y SMTP (1025). La API se ejecuta con el perfil local HTTP http://localhost:5087 o HTTPS https://localhost:7274. Para detener la infraestructura: docker compose -f compose.local.yaml down.

20.3. Production

El entorno de producción levanta la API y la infraestructura en Compose. Se prepara .env.prod a partir de .env.prod.example y se ejecuta: docker compose --env-file .env.prod -f compose.prod.yaml up -d --build. La API queda disponible en http://localhost:5000 y MailDev web en http://localhost:1081. Dentro de la red Docker, la API utiliza postgres:5432, mongodb:27017 y redis:6379. Para detenerlo se utiliza el mismo comando con down en lugar de up -d --build.

20.4. Separación y persistencia

Development y Production utilizan proyectos Compose, redes y volúmenes independientes, por lo que pueden ejecutarse simultáneamente sin compartir datos. Los volúmenes conservan la información de PostgreSQL, MongoDB y Redis. El comando docker compose down no elimina los volúmenes; no se debe utilizar down -v salvo que se quiera borrar expresamente la persistencia. Los archivos .env y .env.prod reales no deben subirse al repositorio si contienen secretos.

20.5. Variables de entorno y seguridad

El repositorio incluye .env.example y .env.prod.example como plantillas. En Production, la configuración JWT se suministra mediante variables como JWT_KEY, JWT_ISSUER, JWT_AUDIENCE, JWT_ADMIN_USERNAME y JWT_ADMIN_PASSWORD. Las credenciales de demostración (admin / Admin123!) y las claves de desarrollo no deben reutilizarse en producción; se deben definir secretos propios y seguros.

21. Inicialización, dependencias y pruebas

21.1. Seeders y comportamiento por entorno

La inicialización incluye CategoriaSeeder, UsuarioSeeder, ProveedorSeeder y FunkoSeeder. Las categorías de ejemplo son Videojuegos, Películas, Series, Anime, Cómics, Música, Deportes y Tecnología. Los proveedores inicializados son Grupo Cladellas, Extended Play y Kilumio; los Funkos de ejemplo incluyen Spider-Man, Batman, Naruto Uzumaki, Darth Vader, Geralt de Rivia y Eleven. En Development se reinicializan las bases de datos y Redis antes de ejecutar los seeders. En Production se conservan los datos y se ejecutan los seeders sin borrado previo.

21.2. Inyección de dependencias

Infrastructure/DependencyInjection.cs centraliza el registro de los servicios de categorías, proveedores y Funkos, CatalogoService, AuthService, correo, JWT, autorización, SignalR, GraphQL, Swagger y CORS. Infrastructure/Persistence/PersistenceExtensions.cs registra AppDbContext con PostgreSQL, IDistributedCache con Redis y los componentes IMongoClient, IMongoDatabase e IMongoCollection<Proveedor> para MongoDB. Las dependencias se reciben mediante los constructores.

21.3. Pruebas y ejecución

El proyecto ApiFunkosConjunta.Tests organiza pruebas de controladores, servicios, repositorios, mappers, seeders, infraestructura, GraphQL, SignalR y correo. NUnit ejecuta los tests, Moq aísla dependencias y FluentAssertions expresa las comprobaciones. Testcontainers utiliza PostgreSQL 18 y MongoDB 8 reales para las pruebas de repositorio; Docker debe estar iniciado. Las pruebas SMTP necesitan MailDev disponible en localhost:1025.

21.4. Comandos de pruebas y cobertura

Para ejecutar el proyecto de pruebas: dotnet test ApiFunkosConjunta.Tests/ApiFunkosConjunta.Tests.csproj. Para recopilar cobertura: dotnet test --settings coverage.runsettings --collect:"XPlat Code Coverage". La cobertura obtenida depende de la ejecución concreta de la suite y del entorno.

22. Conclusión

ApiFunkosConjunta implementa una API para administrar Funkos, categorías y proveedores mediante ASP.NET Core y .NET 10. Combina PostgreSQL, MongoDB y Redis, y organiza la solución en controladores, servicios y repositorios. CatalogoService coordina las reglas que afectan a varias entidades y la invalidación de caché.

La solución incorpora filtros, paginación, autenticación JWT por roles, validación de relaciones y errores tipados. Además, integra Swagger/OpenAPI, GraphQL, SignalR y un servicio de correo electrónico con MailKit. Docker Compose separa los entornos de desarrollo y producción, mientras que NUnit y Testcontainers permiten probar los componentes y la persistencia.

En conjunto, el proyecto proporciona una base organizada y ampliable para la gestión del catálogo, con persistencia híbrida, comunicación en tiempo real, correo, documentación interactiva y pruebas automatizadas.